USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

microSD over SPI, and a Pad Table That Is Not in Pin Order

UM-NATOS-020

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-020	1.0 · 2026-08-15	**Complete, verified on hardware**	Internal

1. Abstract

The first storage in this project that a person can remove, carry away, and fill on another machine. Flash holds what the kernel was built with (UM-NATOS-018); this holds what the user brought.

The driver was straightforward. One constant was not: the ESP32's IO_MUX pad table is **not in pin order**, and the two UART0 pads sit between GPIO22 and GPIO23. Counting up from a known entry put GPIO23 at the address of `U0RXD`, so initialising the SD bus reconfigured the console's receive pin as a GPIO output. Transmit kept working, so the board went on printing telemetry and simply stopped answering.

This report also records the check that *should* have caught that and did not, because it is the more useful half.

2. Why SPI mode

The card supports a 4-bit parallel protocol several times faster. This driver uses 1-bit SPI mode instead:

- it is the mode every card must support, so a card that fails here is faulty rather than unsupported
- the board wires the slot to ordinary GPIOs, not to the ESP32's dedicated SDMMC pins, so the fast path is not available here anyway
- three SPI devices have now been brought up in this project, and reusing a shape that is understood beats learning a protocol whose failure modes are new

2.1 Bit-banged first

As with the display (UM-NATOS-015 §3), the first version bit-bangs at ~250 kHz. Card identification is a one-time cost measured in milliseconds and is *required* to run at 400 kHz or slower, which no hardware peripheral makes easier.

Moving sector reads to SPI3 is a measured optimisation to be taken once something is proven to be reading the right bytes. Moving first would mean debugging a protocol and a peripheral simultaneously, which is what cost the display three commits for one defect.

3. One error code per stage

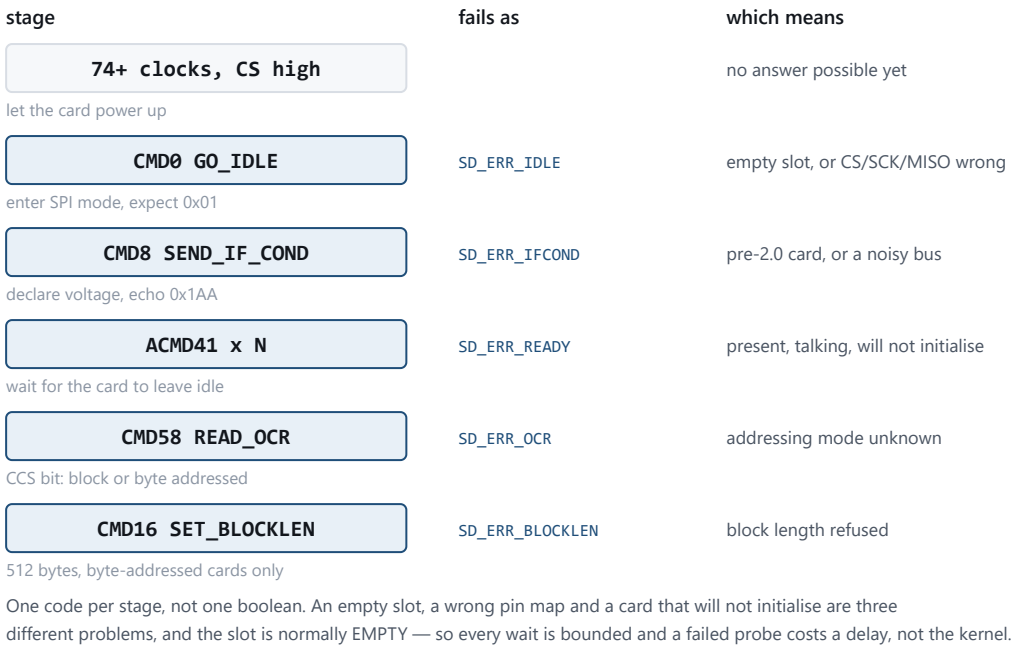

Figure 1. *Card bring-up, and the error code each stage answers to. An empty slot and a wrong pin map produce identical silence, so the stage that failed is the diagnosis.*

`sd_init()` returns a distinct code per stage rather than a boolean. An empty slot, a wrong pin map, a pre-2.0 card and a card that will not complete initialisation are four different problems, and a driver that returns −1 for all of them makes the user guess.

Every wait is bounded, and that is not defensive habit — the slot is normally *empty*. A probe that hangs on an absent card would be worse than no driver at all, because the normal case would be the fatal one. Verified:

```
sd_init FAILED  type=none  last R1=0x000000ff
stage: CMD0 - no card, or MISO/CS/SCK wrong
reporter lines after a failed probe: 4      <- kernel still running
```

`R1 = 0xFF` is unambiguous: bit 7 of a real R1 is always zero, so "no answer" cannot be confused with any legal response.

4. The pad table

`gpio_out_init()` takes a pin number *and* its IO_MUX register address, because the table is not in pin order. The relevant stretch:

```
0x7C GPIO21    0x80 GPIO22    0x84 U0RXD(GPIO3)    0x88 U0TXD(GPIO1)    0x8C GPIO23
```

Counting up from GPIO21 while forgetting the two UART pads puts GPIO23 at `0x84`, which is the console's **receive** pad. Configuring it as a GPIO output kills serial input.

4.1 The symptom named the fault

Transmit is a different pad (`U0TXD` at `0x88` , untouched), so the board kept printing its telemetry at full rate and simply stopped responding to anything typed. A shell that echoes nothing looks like a hung shell; the reporter output proved the kernel was fine, which narrowed it to the input path immediately.

That asymmetry was luck. Had the mistake landed on `0x88` instead, the board would have gone silent and looked like a boot failure.

4.2 The verification that agreed with the wrong answer

The pin map was checked before flashing, against the four IO_MUX entries already in `gpio.h` :

entry	offset
GPIO2	0x40
GPIO12	0x34
GPIO14	0x30
GPIO21	0x7C

All four confirmed the indexing. **All four are below the UART pads**, so every one of them agreed with the wrong answer. The check was real, it was performed, and it could not have failed.

> A cross-check only tests what its samples can distinguish. Four entries that all sit on the same side of the anomaly confirm the rule and say nothing about the exception. The samples must straddle the thing being verified.

This is the same shape as UM-NATOS-017 §7.1, where a calibration inferred axis direction from a sample that could only ever give one answer.

5. Verification

5.1 Block 0 proves the bus, not the addressing

```
sdread 0 -> FA 33 C0 8E D0 BC 00 7C 8B F4 50 07 50 1F FB FC
            BF 00 06 B9 00 01 F2 A5 EA 1D 06 00 00 BE BE 07
            signature=0x55 0xAA
            partition 0: type=0x06 start=240 sectors=490000
```

`FA 33 C0 8E D0 BC 00 7C` is the textbook MBR bootstrap — `cli` , `xor ax,ax` , `mov ss,ax` , `mov sp,0x7C00` — and `BE BE 07` points at the partition table. Real data, correctly framed.

It proves nothing about the **addressing mode**, and that is worth stating because it looks like a complete result. Block 0 is byte 0 on a byte-addressed card and block 0 on a block-addressed one. The read succeeds either way and discriminates nothing.

5.2 A filesystem header at a non-zero LBA does

```
sdread 240 -> signature=0x55 0xAA
              fs type: FAT16
```

Those five characters require the pin map, the clock, the addressing mode and the block framing to all be correct **simultaneously**. A wrong addressing mode would land 512 times too far into the card and return something that is not a filesystem header.

The card is ~250 MB and genuinely `SDSC`, which was luck: it exercised the byte-addressing path that a modern card would have skipped entirely.

6. Metrics

Quantity	Value
Mode	SPI, 1-bit, bit-banged
Identification clock	~250 kHz (spec requires ≤400 kHz)
Error codes	7, one per stage
Unbounded waits	0
Card under test	~250 MB, SDSC, FAT16
Partition found	type 0x06, start LBA 240, 490,000 sectors
IO_MUX entries cross-checked	4
IO_MUX entries that could have caught the fault	0

7. What this does not establish

- **No write path.** `sd_read_block()` only. Nothing writes to the card.
- **No filesystem.** The MBR is decoded far enough to find a partition and confirm a FAT signature. There is no directory walk, no file lookup, no FAT chain traversal.
- **No card-removal detection.** Pull the card mid-read and the result is a token timeout rather than a clean "card gone" error. Nothing polls for insertion either.
- **SDHC is untested.** The `CCS` bit is read and the block/byte addressing branch exists, but the only card available was SDSC. The path that a modern card would take has never executed.
- **Still slow.** Bit-banged at every stage including bulk reads. Moving to SPI3 is deliberate future work, not an oversight, but nothing here measures what the current throughput actually is.
- **The pin map is confirmed only for this board.** A different ESP32 board wiring the slot elsewhere would need the four constants rederived, and §4.2 is the warning about how to check them.

8. References

- UM-NATOS-015 §3 — bit-banging first, and why

- UM-NATOS-017 §7.1 — the other verification that could only give one answer
- UM-NATOS-018 — the flash record, the other half of storage
- `kernel/sd.c` — `spi1_xfer` equivalent, the pad table note, per-stage codes
- `kernel/sd.h` — mode reasoning and the error enum